



Problem Solving With C - UE24CS151B

File IO – Using Redirection Operator

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

File IO – Using Redirection Operator



1. Introduction
2. Execution Steps
3. Sample C code execution – User's Input
4. Sample C code execution – File Input
5. Disadvantages

PROBLEM SOLVING WITH C

File IO – Using Redirection Operator



Introduction

- A program in C allows **reading from or writing to files** instead of standard input/output (keyboard/screen).
- The **< operator is used in the command line** (not in C code) to **redirect input** from a file to the program's stdin.
- To feed input data from a file to a source code without changing the source code.
- Very useful in **Testing and Automation**.

PROBLEM SOLVING WITH C

File IO – Using Redirection Operator



Execution Steps

- `./a.out < input.txt` OR `a.exe < input.txt`
- Run the program and take input from input.txt instead of the keyboard.
- If the file doesn't exist, the shell throws an error and the program does not run.
- Input redirection operator (<) is handled before the program starts.

PROBLEM SOLVING WITH C

File IO – Using Redirection Operator



Sample C code Execution: Input given by the user once the code is run

```
#include <stdio.h>

int main() {
    int num;
    printf("Reading numbers from input...\n");
    while (scanf("%d", &num) != EOF) {
        printf("Read number: %d\n", num);
    }
    return 0;
}
```

```
C:\Users\Dell>a
Reading numbers from input...
56
Read number: 56
-6
Read number: -6
34
Read number: 34
Out of application
```

PROBLEM SOLVING WITH C

File IO – Using Redirection Operator



Sample C code Execution: Input given through the file numbers.txt once the code is run. Use < and the file must be existing.

```
#include <stdio.h>

int main() {
    int num;
    printf("Reading numbers from input...\n");
    while (scanf("%d", &num) != EOF) {
        printf("Read number: %d\n", num);
    }
    return 0;
}
```

```
C:\Users\Dell>a < numbers.txt
Reading numbers from input...
Read number: 3
Read number: 23
Read number: 44
Read number: 77
Read number: -15
Read number: 40
Read number: -266
Read number: 78
Read number: 22
Read number: 66
Read number: 90
Read number: 189
Out of application
```

PROBLEM SOLVING WITH C

File IO – Using Redirection Operator



Disadvantages

- **No Error Handling in Code:** If the file doesn't exist, your program won't even start. You can't check or handle the error inside your code.
- **Limited to Standard Input (stdin):** Only redirects stdin. Can't easily control multiple files or write to output files with this method.
- **Not Portable Across All Systems:** Works in Unix/Linux-based terminals, but may not behave the same in some IDEs or Windows environments.
- **Not Flexible for Dynamic File Names:** File name must be known and provided at command line.



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

Prof. Sindhu R Pai - sindhurpai@pes.edu



Problem Solving With C - UE24CS151B

File Handling in C

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

File Handling



1. Points to Discuss
2. Introduction
3. Need of Files
4. File Classification
5. Operations on Files
6. Read/Write Operations on Files

PROBLEM SOLVING WITH C

File Handling



Points to Discuss!!!

- Can we use the data entered by the user in one program execution, in another program execution without asking the user to enter again?
- How to generate same input several times?
- How to store the output produced for future references?
- Program stores the result what if one wants to store other data too?
- How to categorize and manage forms of data produced?

PROBLEM SOLVING WITH C

File Handling



Introduction

- Variables used in program will die at the end of execution
- To persist data even after the program execution is complete, use Files
- File represents a sequence of bytes and it is a source of storing information in sequence of bytes on a disk
- The data in a file can be structured or unstructured
- A program in C can itself be the data file for the program
- The keyboard and the output screen are also considered as files

PROBLEM SOLVING WITH C

File Handling



Need of Files

- When a program is terminated, the entire data is lost. Storing in a data file will preserve the data even if the program terminates.
- If the data is too large, a lot of time spent in entering them to the program every time the code is run.
- If stored in a data file, easier to access the contents of the data file using few functions in C

File Classification

- **Text File**

- Contains textual information in the form of alphabets, digits and special characters or symbols
- Created using a text editor

- **Binary File**

- Contain bytes or a compiled version of a text file i.e. data in the form of 0's and 1's
- Can store larger amount of data that are not readable but secured.

PROBLEM SOLVING WITH C

File Handling



Operations on Files

- Reading the contents of the file
- Writing the contents to the file

Note: To perform any operation on Files, **The physical filename, the logical filename and the mode** must be connected using **fopen()**

- **Physical Name:** A file is maintained by the OS. The OS decides the naming convention of a file
- **Logical Name:** In a C Program, identifier is used to refer to a file. Also called as **File Handle**
- **Mode:** Can be read only, write only, append or a combination of these.

PROBLEM SOLVING WITH C

File Handling



C Functions to perform Operations

- Creation of new file or open an existing file using **fopen()**
- Read operation on a file using **fgetc()**, **getc()**, **fscanf()** and **fgets()**
- Write operation on a file using **fputc()**, **putc()**, **fprintf()** and **fputs()**
- Moving to a specific location in a file using **fseek()**, **ftell()** and **rewind()**
- Closing a file using **fclose()**

PROBLEM SOLVING WITH C

File Handling



fopen()

Syntax: `fopen("path of the file with filename", "mode");`

`// mode can be r, w, a, r+, w+, a+`

- Opens a file in the specified mode and returns a FILE pointer(address of the structure which contains information about the attributes of the file) if it succeeds. Else returns NULL.
- The reasons for failure in file opening:
 - File might not be available
 - File might not have permission to open it in the mode specified.
- Initializing a FILE pointer does not mean that the whole file is made available in memory. Other functions are required to access the contents

PROBLEM SOLVING WITH C

File Handling



`fclose()`

- Closes the stream. All buffers are flushed
- Returns zero if the stream is successfully closed. On failure, EOF is returned
- All links to the file are broken
- Misuse of files is prevented

Syntax: `fclose(file_pointer);`

- File pointer can be reused
- Coding Examples

PROBLEM SOLVING WITH C

File Handling



Read /Write operations on File

- Categories
 - **Character read/write**
 - `fputc()` , `fgetc()`, `getc()` and `putc()`.
 - **String read/write**
 - `fgets()` and `fputs()`.
 - **Formatted read/write**
 - `fscanf()` and `fprintf()`.
 - **Block read/write**
 - `fread()` and `fwrite()`

PROBLEM SOLVING WITH C

File Handling



Character I/O operations on File

- Reads a character from the file and increments the file pointer position.
 - **Syntax:** `int fgetc(FILE *fp);`
 - **Return Value:** Next byte from the input stream on success, EOF on error.
- Write operation at current file position and increments the file pointer position.
 - **Syntax:** `int fputc(int c, FILE *fp);`
 - **Return Value:** Character that is written on success, EOF on error.
- Coding Examples

PROBLEM SOLVING WITH C

File Handling



String I/O Operations on File(fgets() and fputs())

- Reads a line of characters from file and stores it into the string pointed to by char_array variable. It stops when either (n-1) characters are read, the newline character is read, or the end-of-file is reached, whichever comes first
 - **Syntax:** `char* fgets(char *char_array, int n, FILE *stream)`
 - **Return Value:** Pointer to the string buffer on success, NULL on EOF or Error.
- Write a line of characters to a file.
 - **Syntax:** `int fputs(const char *s, FILE *stream)`
 - **Return Value:** A non-negative number on success, EOF on error.
- Coding examples

PROBLEM SOLVING WITH C

File Handling



Formatted Read/Write Operations on File(fscanf(),fprintf())

- Reads the formatted data from the file instead of standard input.
 - **Syntax:** `int fscanf(FILE *fp, const char *format[,address,.....]);`
 - **Return Value:** The number of **values read** on success ,**EOF** on failure.
- Writes the formatted data to a file instead of standard output.
 - **Syntax:** `int fprintf(FILE *fp, const char *format[,argument,.....]);`
 - **Return value:** The number of **characters written** on success , **EOF** on failure.
- Coding Examples

Block Read/Write Operations on a File

- Reads an entire block from a given file.
 - **Syntax:** `size_t fread(void *p, size_t size, size_t n, FILE *fp);`
 - **Return Value:** The number of values successfully read , error or zero for size ≤ 0 .
- Writes an entire block to the file.
 - **Syntax:** `size_t fwrite(const void *p, size_t size, size_t n, FILE *fp);`
 - **Return Value:** The number of values successfully written, error or zero for size ≤ 0 .

PROBLEM SOLVING WITH C

File Handling



Random access to a file

- Seeking the pointer position in the file at the specified byte.
 - **Syntax:** `fseek( FILE* pointer, long offset, int whence)`
 - **Return Value:** zero if successful, or else it returns a non-zero value.
- The function returns the current pointer position ,value from the beginning of file.
 - **Syntax:** `ftell(FILE* pointer)`
 - **Return Value:** 0 or a positive integer on success and -1 on error.
- The function is used to move the file pointer to the beginning
 - **Syntax:** `rewind(FILE* pointer)` // Function does not return anything.



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

Prof. Sindhu R Pai - sindhurpai@pes.edu



PROBLEM SOLVING WITH C UE20CS151

Prof. Pushpa G and Prof. Sindhu R Pai

Department of Computer Science and Engineering



Problem Solving With C - UE24CS151B

Error Handling in File Handling

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

Error Handling in File Handling



1. Common Programming Errors
2. Error Handling
3. Demo of Errors
4. Best Practices

PROBLEM SOLVING WITH C

Error Handling in File Handling



Common Programming Errors that occur during File I/O

- Open with invalid filename
- Operation on a file that has not been opened
- Operation not permitted by 'fopen'
- Write to a write-protected file
- Read beyond end-of-file

PROBLEM SOLVING WITH C

Error Handling in File Handling

Error Handling

- Not supported by C directly and is accomplished by using **errno.h**
- When a function is called, a global variable named as **errno** is automatically assigned a value used to identify the type of error that has been encountered

errno value	Type of Error
1	Operation not permitted
2	No such file or directory
5	I/O error
7	Argument list too long
9	Bad file number
11	Try again
12	Out of memory
13	Permission denied

- Coding Examples using `perror()` and `strerror()`

PROBLEM SOLVING WITH C

Error Handling in File Handling



Error Handling continued..

- **Two-status library functions** are used to prevent performing any operation beyond EOF
- **feof():** Used to test for an end of file condition
 - **Syntax:** `feof(FILE *file_pointer);`
 - **Return Value:** Non-zero if EOF, Else zero
- **ferror():** Used to check for the error in the stream
 - **Syntax:** `int ferror (FILE *file_pointer);`
 - **Return Value:** Non-zero if error occurs, Else zero
- The error indication will last until the file is closed(`fclose()`) or cleared by the `clearerr()` function.

PROBLEM SOLVING WITH C

Error Handling in File Handling

Demo of Error Handling

- C code demonstration of Error handling in File I/O



PROBLEM SOLVING WITH C

Error Handling in File Handling



Best Practices

- Given a file pointer check whether it is **NULL before proceeding with further operations.**
- Use **errno.h** and **global variable errno** to know the type of error that occurred. Usage of **strerror()** and **perror()** helps in providing textual representation of the current errno value.
- Good to check **whether EOF is reached or not before performing any operation on the file.**



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

Prof. Sindhu R Pai - sindhurpai@pes.edu



Problem Solving With C - UE24CS151B

Searching on File Contents

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

Searching on File Contents



1. Data file Description
2. Problem Statement
3. Requirements to solve the Problem
4. Demo of C Solution

PROBLEM SOLVING WITH C

Searching on File Contents



Data File Description

- Data file to be used: **numbers.txt**
- Contains a collection of integers in an **ascending order**.
- All integers are separated by white space in between.

PROBLEM SOLVING WITH C

Searching on File Contents



Problem Statement

- Create a program to perform Binary Search on the collection of integers stored in the file. Use numbers.txt to extract the dataset. Ask the user to enter the key integer to be searched in the file. Display appropriate message to the user.

PROBLEM SOLVING WITH C

Searching on File Contents



Requirements to solve the problem

- Create an array of integers of very big size.
- Reading the text file integer by integer for n integers.
- Store these integers in the array created initially.
- Compare the integer in the array with the key element entered by the user using the Binary Search Algorithm.
- Display appropriate message.

PROBLEM SOLVING WITH C

Searching on File Contents



Demo of C Solution

- Demonstration of C Code

PROBLEM SOLVING WITH C

Searching on File Contents



Try It!!

- Use some csv file containing many fields and apply callback function based on the field to be searched.



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

Prof. Sindhu R Pai - sindhurpai@pes.edu



Problem Solving With C - UE24CS151B

Sorting the File Contents

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

Sorting the File Contents



1. Data file Description
2. Problem Statement
3. Requirements to solve the Problem
4. Demo of C Solution

PROBLEM SOLVING WITH C

Sorting the File Contents



Data File Description

- Data file to be used: **student.csv**
- Contains the information of students in two columns only.
- First row represents the column headers such as **roll_no** and **name**.
- Contains around 55 rows. Every data is stored with a **comma(,)** in between.

PROBLEM SOLVING WITH C

Sorting the File Contents



Problem Statement

- Create a menu driven program to perform selection sort based on roll_number and name. Use student.csv to extract the dataset and write the sorted record to a new file.

PROBLEM SOLVING WITH C

Sorting the File Contents



Requirements to solve the problem

- Creating the student type with two data members – roll_no and name
- Reading the csv file line by line
- Splitting the line based on the delimiter using strtok function
- Copying the data from the data file to the data members of the student structure
- Client code requires menu driven code - Provide the option to the user to sort based on roll_no or name
- Perform selection sort based on the option from the user
- Create a new file using fopen and sorted record write to a new file(roll_no+name)

PROBLEM SOLVING WITH C

Sorting the File Contents



Demo of C Solution

- Demonstration of C Code

PROBLEM SOLVING WITH C

Sorting the File Contents



Try It!!

- Use callback function and avoid writing two different functions.



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

Prof. Sindhu R Pai - sindhurpai@pes.edu



Problem Solving With C - UE24CS151B

Header Files – Creation and Usage

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

Header Files – Creation and Usage



1. Introduction
2. Why Header files?
3. Creating a Header file and Using a Header File
4. Implementation File
5. Execution Steps

Introduction

- Play a crucial role in **organizing code and promoting reusability**.
- Typically contains **function declarations, macro definitions, structure declarations, and constants** that can be shared across multiple source files.
- This allows developers to separate the **interface** from the **implementation**, leading to cleaner, more modular programs.

Why use Header files?

- **Code reusability:**

Write once, use in multiple programs.

- **Improved readability:**

Keeps the main file clean and focused.

- **Ease of maintenance:**

Updates made in the header reflect across all included files.

PROBLEM SOLVING WITH C

Header Files – Creation and Usage



Creating a Header File

- Create a file with the .h extension (e.g., mathutils.h).
- Write function declarations, macros, or global variables inside.
- Use include guards (#ifndef, #define, #endif) to prevent multiple inclusions.

```
// mathutils.h
```

```
#ifndef MATHUTILS_H  
#define MATHUTILS_H
```

```
int add(int a, int b);  
float squareRoot(float x);
```

```
#endif
```

PROBLEM SOLVING WITH C

Header Files – Creation and Usage



Using a Header File

```
// main.c
```

```
#include <stdio.h>
```

```
#include "mathutils.h"
```

```
int main() {
```

```
    printf("Sum: %d\n", add(5, 3));
```

```
    printf("Square root of %d is %f", 25, squareRoot(25));
```

```
    return 0;
```

```
}
```

PROBLEM SOLVING WITH C

Header Files – Creation and Usage



Implementation file

- Define functions in a separate .c file (e.g., mathutils.c)

```
// mathutils.c
```

```
#include "mathutils.h"
int add(int a, int b) {
    return a + b;
}
float squareRoot(float x){
    return sqrt(x);
}
```

PROBLEM SOLVING WITH C

Header Files – Creation and Usage



Execution Steps

- **Compiling separately into object files:**

```
gcc -c main.c    // creates main.o
```

```
gcc -c mathutils.c // creates mathutils.o
```

- **Linking:**

```
gcc main.o mathutils.o -o myprogram
```

- **Execution:**

```
./myprogram    OR    myprogram.exe
```



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

Prof. Sindhu R Pai - sindhurpai@pes.edu



Problem Solving With C - UE24CS151B

Environment Variables

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

Environment Variables



- Introduction
- Standard Environment Variables
- Environment Access
- Demo of C code

PROBLEM SOLVING WITH C

Environment Variables



Introduction

- A set of dynamic-named values that can affect the way the running processes behave on a computer
- They are part of the environment in which the process runs
- When a program is executed, it receives **information about the context in which it was invoked** using two ways -> **CLA and Environment variables**

```
int main(int argc,char*argv[],char*envp[])
```

- Programs executed from the shell inherit all of the environment variables from the shell

PROBLEM SOLVING WITH C

Environment Variables



Standard Environment Variables

- Used for information about the **user's home directory, terminal type, current locale** and etc.
- The set of all environment variables that have values is collectively known as the ***environment***
- Names of environment variables are **case - sensitive** and must contain the character '='
- The values of environment variables can be anything that can be represented as a string. **Must not contain an embedded null character**

PROBLEM SOLVING WITH C

Environment Variables



Environment Access

- **char * getenv (const char *name)**
 - Returns a string that is the value of the environment variable name
 - If the environment variable name is not defined, the value is a NULL pointer
- **int putenv (char *string)**
 - Adds or changes the value of the environment variable
 - String is of the form '**name=value**'. If name is not there in the environment, string added to the environment. If name exist, name is updated with the new value
 - Returns 0 on success and else Non-zero and errno is set

PROBLEM SOLVING WITH C

Environment Variables



Environment Access continued..

- **int setenv (const char *name, const char *value, int replace)**
 - Used to add a new definition to the environment
 - On success, return 0. Else, environment is unchanged with -1 return value and errno is set
- **int unsetenv (const char *name)**
- **int clearenv (void)**
- **char ** environ**
 - Represented as an array of strings. Each string is of the format '**name=value**'
- All these functions are available in **stdlib.h**

PROBLEM SOLVING WITH C

Environment Variables



Demo of C Code

- **Code to display the Environment variable and setting the Environment variable**



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

Prof. Sindhu R Pai - sindhurpai@pes.edu



Problem Solving With C - UE24CS151B

Preprocessor Directives

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

Preprocessor Directives



- Introduction
- Types of Pre-processor Directives
- Macros
- Points to know about Macros
- Predefined Macros
- Macro vs Enum
- File Inclusion Directives
- Conditional compilation Directives
- Other Directives

PROBLEM SOLVING WITH C

Preprocessor Directives



Introduction

- **The lines of code** in a C program which are **executed by the 'C' pre-processor**
- A **text substitution tool** and instructs the compiler to do required pre-processing before the actual compilation
- All pre-processor commands begin with a hash symbol (#)
- It must be the first nonblank character and for readability, pre-processor directive should begin in the first column conventionally

PROBLEM SOLVING WITH C

Preprocessor Directives



Types of Preprocessor Directives

- Types include:
 - Macros
 - File Inclusion
 - Conditional Compilation
 - Other directives
- Possible to see the effect of the pre-processor on source files directly by using the **-E option of gcc**

PROBLEM SOLVING WITH C

Preprocessor Directives



Macros

- A piece of code in a program which has been given a name
- During preprocessing, it substitutes the name with the piece of code
- **#define** directive is used to define a macro.
- Example: `#define PI 3.14`
- Coding Examples

PROBLEM SOLVING WITH C

Preprocessor Directives



Points to know about Macros

- Macro **does not** judge anything
- **No memory Allocation** for Macros
- Can define string using macros
- Can define macro with expression
- Can define macro with parameter
- Macro can be used in another macro
- Constants defined using #define cannot be changed using the assignment operator
- Redefining the macro with #define is allowed. But not advisable

PROBLEM SOLVING WITH C

Preprocessor Directives

Predefined Macros



S.no	Macro & Description
1	__DATE__ The current date as a character literal in "MMM DDYYYY" format.
2	__TIME__ The current time as a character literal in "HH:MM:SS" format.
3	__FILE__ This contains the current filename as a string literal.
4	__LINE__ This contains the current line number as a decimal constant.
5	__STDC__ Defined as 1 when the compiler complies with the ANSI standard.

PROBLEM SOLVING WITH C

Preprocessor Directives



Macro vs Enum

- Macro doesn't have a type and enum constants have a type int.
- Macro is substituted at pre-processing stage and enum constants are not.
- Macro can be redefined using `#define` but enum constants cannot be redefined. However assignment operator on a macro results in error

PROBLEM SOLVING WITH C

Preprocessor Directives



File Inclusion Directives

- Instructs the pre-processor to include a file in the program using **#include** directive
- Two types of files
 - **Header File or Standard files: Included between < and >**
 - Contains the definition of pre-defined functions like printf(), scanf() etc.
 - To work with these functions, header files must be included
 - **User defined files: Included using “ and “**
 - When a program becomes very large, it is good practice to divide it into smaller files and include whenever needed.
 - Adding user defined header files in the program

PROBLEM SOLVING WITH C

Preprocessor Directives



Conditional Compilation Directives

- A few blocks of code will be compiled in a particular program based on the result of some condition
- Conditions can be mentioned using - **#ifdef, #ifndef, #if, #else, #elif, #else, #endif**
- Coding Examples

PROBLEM SOLVING WITH C

Preprocessor Directives



Other Directives

- **#undef Directive:** Used to undefine an existing macro
 - Example: #undef LIMIT
 - After this statement every “#ifdef LIMIT” statement will evaluate to false
- **#pragma startup and #pragma exit:** Helps to specify the functions that are needed to run before program startup and just before program exit
- **#pragma warn:** This directive is used to hide the warning messages which are displayed during compilation using **-rvl**, **-par** and **-rch**



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU
Prof. Sindhu R Pai - sindhurpai@pes.edu



Problem Solving With C - UE24CS151B

Portable Program Development

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

Portable Program Development



- Problem Statement
- Requirements
- Demo of C Solution

PROBLEM SOLVING WITH C

Portable Program Development



Problem Statement

- Demonstrate the real use of conditional compilation and pre-defined macros. Make the code portable across different platforms. Based on the current platform the code is run, particular part of the code must be compiled and executed

PROBLEM SOLVING WITH C

Portable Program Development



Requirements

- Part of the code to be compiled and executed depends on the macro set or not

If **mingw in windows system**, the value of **__MINGW32__** will be 1

If **unix/Linux system** is used, the value of **__unix__** is 1

If **mac system** is used, the value of **__APPLE** is 1

Note that there is no **{}** to specify the block

No usage of **()** to specify the macro

PROBLEM SOLVING WITH C

Portable Program Development



Demo of C Solution

- See the output of preprocessing on the terminal using `gcc -E` and then execute the code



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU
Prof. Sindhu R Pai - sindhurpai@pes.edu